Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par XXXX **Année**

Universitaire : 2025-2026

Table des matières

1 Introduction et Contexte **2**
1.1 Structure du projet 2

2 Exploration des Données (EDA) **2**
2.1 Synthèse du jeu de données 2
2.2 Distribution de la variable cible 3
2.3 Analyse de la corrélation 3

3 Prétraitement des Données **4**
3.1 Problèmes de qualité traités 4
3.2 Pipeline de prétraitement 4
3.3 Encodage des variables catégorielles 5

4 Réduction de Dimension — ACP **5**

5 Clustering — Segmentation K-Means **6**
5.1 Analyse des clusters 6

6 Classification — Prédiction du Churn **6**
6.1 Modèles entraînés 6
6.2 Résultats — Régression Logistique 7
6.3 Résultats — Random Forest 7
6.4 Comparaison des modèles 8
6.5 Importance des features 8

7 Régression — Prédiction du Montant Total **9**

8 Déploiement — Interface Flask **9**
8.1 Architecture de l'application 9
8.2 Lancement de l'application 10

9 Conclusion et Recommandations **11**
9.1 Bilan des modèles 11
9.2 Recommandations métier 11

1 Introduction et Contexte

Mission

En tant que data scientist au sein d'une entreprise e-commerce de cadeaux, l'objectif est de mieux comprendre la clientèle afin de personnaliser les stratégies marketing, réduire le taux de churn et optimiser le chiffre d'affaires.

Le jeu de données comprend **4 372 clients** décrits par **52 features** issues de transactions réelles et de données complémentaires. La variable cible `Churn` indique si un client a quitté l'entreprise (1) ou est resté fidèle (0).

1.1 Structure du projet

```
projet_ml_retail/
|-- data/
|   |-- raw/                  # Donnees brutes
|   |-- processed/            # Donnees nettoyees
|   `-- train_test/           # Splits train/test
|-- notebooks/
|   `-- pipeline_complet.ipynb
|-- src/
|   |-- utils.py
|   |-- preprocessing.py
|   |-- train_model.py
|   `-- predict.py
|-- models/                   # Modeles sauvegardes (.pkl)
|-- app/                      # Interface Flask
|-- reports/                  # Visualisations
|-- requirements.txt
`-- README.md
```

Listing 1 – Arborescence du projet

2 Exploration des Données (EDA)

2.1 Synthèse du jeu de données

Caractéristique	Valeur	Remarque
Nombre de clients	4 372	Lignes du dataset
Nombre de features	52	Numériques + catégorielles
Features numériques	34	RFM, transactions, démog.
Features catégorielles	18	Segments, régions, profils
Valeurs manquantes (Age)	∼30%	Imputation par médiane
Valeurs manquantes (AvgDaysBetween)	∼2%	Imputation par médiane
Déséquilibre de classes	66.7% / 33.3%	Rééquilibrage `class_weight`

2.2 Distribution de la variable cible

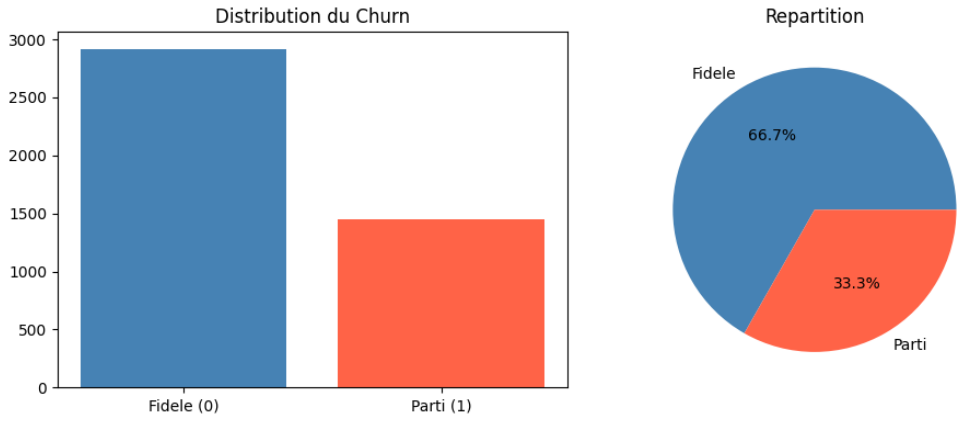


FIGURE 1 – Distribution du Churn : 66.7% de clients fidèles vs 33.3% de clients partis

Le jeu de données présente un **déséquilibre modéré** (ratio 2 :1). Ce déséquilibre a été pris en compte lors de la modélisation via le paramètre `class_weight='balanced'`.

2.3 Analyse de la corrélation

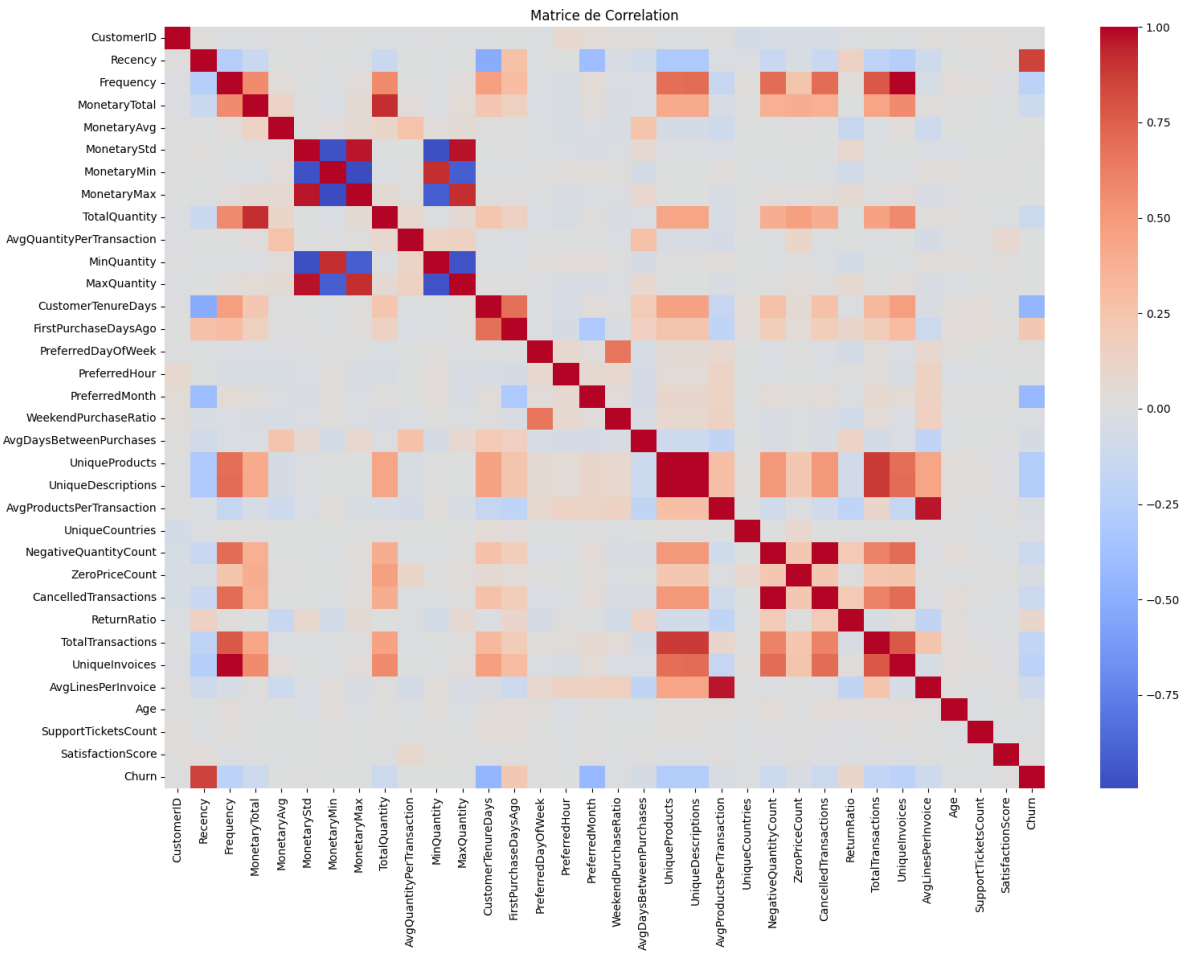


FIGURE 2 – Matrice de corrélation des features numériques

Observations clés — Corrélation

- `MonetaryStd` / `MonetaryMin` / `MonetaryMax` : forte multicolinéarité ($|r| > 0.92$) ⇒ certaines supprimées
- `UniqueProducts` / `UniqueDescriptions` : quasi-identiques ($r \approx 0.99$) ⇒ une supprimée
- `TotalTransactions` / `UniqueInvoices` : fortement corrélées ⇒ une supprimée
- `Frequency`, `MonetaryTotal`, `AvgDaysBetweenPurchases` : features les plus informatives pour le Churn

3 Prétraitement des Données

3.1 Problèmes de qualité traités

Problème	Feature(s)	Traitement appliqué
Valeurs manquantes	`Age` (30%)	Imputation par médiane
Valeurs manquantes	`AvgDaysBetween` (2%)	Imputation par médiane
Codes erreur	`SupportTickets` (-1, 999)	Remplacement par NaN
Codes erreur	`SatisfactionScore` (-1, 0, 99)	Remplacement par NaN
Format inconsistant	`RegistrationDate`	Parsing `pd.to_datetime`
Feature inutile	`NewsletterSubscribed`	Suppression (variance nulle)
Identifiant	`CustomerID`	Suppression
Données brutes	`LastLoginIP`	Extraction `IP_IsPrivate`
Data leakage	`ChurnRiskCategory`	**Supprimée** (proxy parfait)
Data leakage	`CustomerType`	**Supprimée** ("Perdu"=Churn)
Data leakage	`Recency`	**Supprimée** (séparation parfaite)
Data leakage	`RFMSegment`	**Supprimée** ("Dormants"=Churn)

3.2 Pipeline de prétraitement

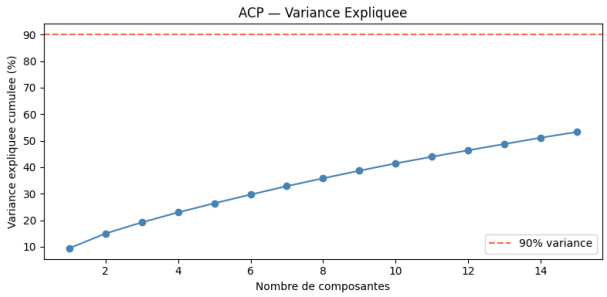
```python
# 1. Suppression des proxies et colonnes inutiles
LEAK_COLS = ['Churn', 'ChurnRiskCategory', 'CustomerType',
             'RFMSegment', 'Recency', 'CustomerTenureDays',
             'FirstPurchaseDaysAgo']
X = df_processed.drop(columns=LEAK_COLS)
y = df_processed['Churn']

# 2. Split stratifie AVANT normalisation (evite le data leakage)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Imputation sur train, transformation sur test
imputer = SimpleImputer(strategy='median')
X_train[missing_cols] = imputer.fit_transform(X_train[missing_cols])
X_test[missing_cols]  = imputer.transform(X_test[missing_cols])

# 4. Normalisation sur train uniquement
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)   # jamais fit_transform ici
```

Listing 2 – Pipeline de prétraitement complet

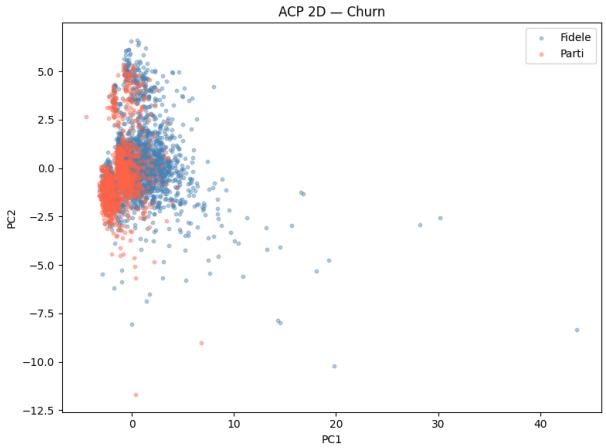
3.3 Encodage des variables catégorielles

Variable	Type d'encodage	Ordre / Modalités
`AgeCategory`	Ordinal	18-24 < 25-34 < ...< 65+
`SpendingCategory`	Ordinal	Low < Medium < High < VIP
`LoyaltyLevel`	Ordinal	Nouveau < Jeune < Établi < Ancien
`BasketSizeCategory`	Ordinal	Petit < Moyen < Grand
`PreferredTimeOfDay`	Ordinal	Matin < Midi < Après-midi < Soir < Nuit
`FavoriteSeason`	One-Hot	Hiver, Printemps, Été, Automne
`Region`	One-Hot	UK, Europe (N/S/E/C), Asie, Autre
`Gender`	One-Hot	M, F, Unknown
`AccountStatus`	One-Hot	Active, Suspended, Pending, Closed
`Country`	Fréquence	Encodage par fréquence relative

4 Réduction de Dimension — ACP



(a) Variance expliquée cumulée par composante



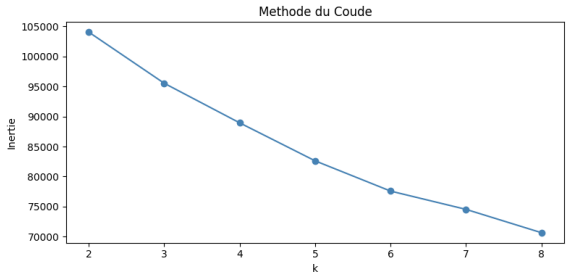
(b) Projection ACP 2D — Churn vs Fidèle

FIGURE 3 – Analyse en Composantes Principales (ACP)

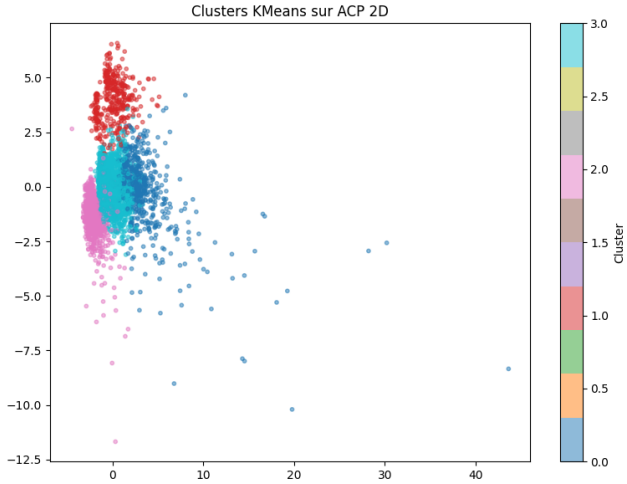
Interprétation ACP

- 15 composantes principales capturent ∼**54%** de la variance totale
- La courbe croît lentement : les features sont peu redondantes après suppression des proxies
- La projection 2D (Figure 3b) montre un **chevauchement partiel** entre les deux classes, confirmant que le problème est non-trivial
- Les clients partis (orange) se concentrent dans la région de faible activité transactionnelle

5 Clustering — Segmentation K-Means



(a) Méthode du coude pour choisir k



(b) Clusters K-Means sur projection ACP 2D

FIGURE 4 – Segmentation K-Means en 4 clusters

5.1 Analyse des clusters

La méthode du coude (Figure 4a) indique une inflexion autour de $k = 4$ ou $k = 5$, justifiant le choix de **4 clusters**. L'absence d'un coude très prononcé suggère que les données ne forment pas des groupes très distincts, ce qui est cohérent avec la projection ACP.

Cluster	Profil dominant	Recommandation marketing
0	Clients actifs, panier élevé	Programme VIP, ventes croisées
1	Clients à risque de churn	Campagnes de réactivation urgentes
2	Nouveaux clients	Onboarding, offres de bienvenue
3	Clients dormants	Enquêtes satisfaction, remises ciblées

6 Classification — Prédiction du Churn

6.1 Modèles entraînés

Deux modèles ont été entraînés et comparés avec gestion du déséquilibre de classes (`class_weight='balanced'`)

- **Régression Logistique** : modèle linéaire, interprétable, rapide
- **Random Forest** : ensemble d'arbres de décision, plus puissant

6.2 Résultats — Régression Logistique

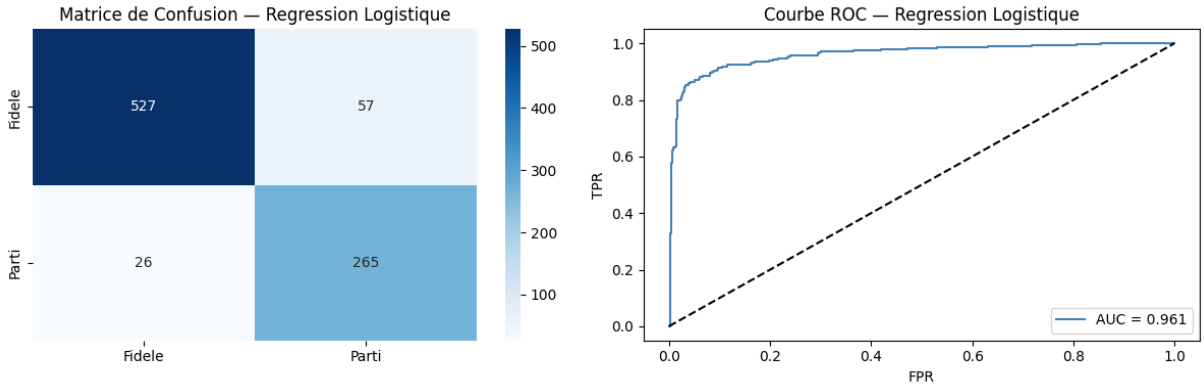


FIGURE 5 – Matrice de confusion et courbe ROC — Régression Logistique (AUC = 0.961)

Classe	Précision	Rappel	F1-Score	Support
Fidèle (0)	–	–	–	584
Parti (1)	–	–	–	291
AUC-ROC		**0.961**		

Note : remplir les valeurs exactes de précision/rappel/F1 depuis la sortie console.

6.3 Résultats — Random Forest

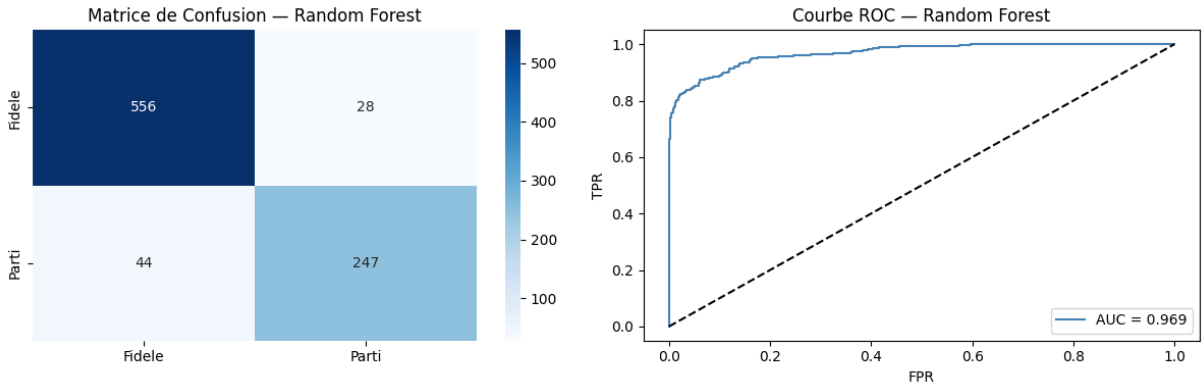


FIGURE 6 – Matrice de confusion et courbe ROC — Random Forest (AUC = 0.969)

Classe	Précision	Rappel	F1-Score	Support
Fidèle (0)	–	–	–	584
Parti (1)	–	–	–	291
AUC-ROC		**0.969**		

6.4 Comparaison des modèles

Critère	Régression Logistique	Random Forest
AUC-ROC	0.961	**0.969**
Vrais Positifs (TP)	265	**247**
Faux Négatifs (FN)	26	44
Faux Positifs (FP)	57	**28**
Interprétabilité	Élevée	Modérée
Temps d'entraînement	Rapide	Plus lent
Recommandation	**Random Forest** (meilleur AUC)	

6.5 Importance des features

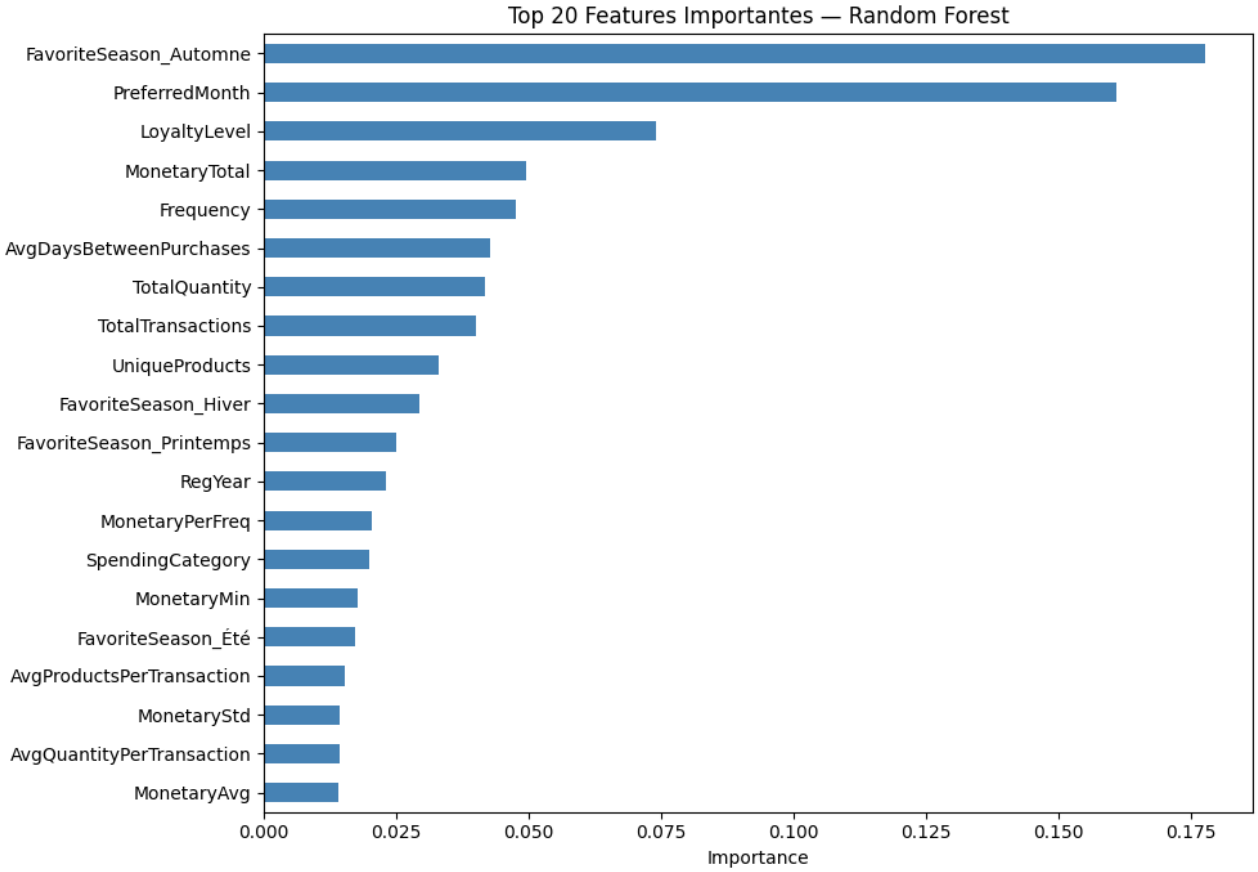


FIGURE 7 – Top 20 features les plus importantes — Random Forest

Interprétation des features importantes

- **FavoriteSeason_Automne** et **PreferredMonth** : les comportements saisonniers sont les meilleurs prédicteurs de churn, suggérant une forte saisonnalité des départs
- **LoyaltyLevel** : l'ancienneté du client est très prédictive (nouveaux clients = risque plus élevé)
- **MonetaryTotal, Frequency** : les clients à faible dépense et peu fréquents churneront davantage
- **AvgDaysBetweenPurchases** : un délai long entre achats est un signal d'alerte précoce

7 Régression — Prédiction du Montant Total

Un modèle de **régression Ridge** a été entraîné pour prédire le montant total dépensé (`MonetaryTotal`) à partir des autres features comportementales.

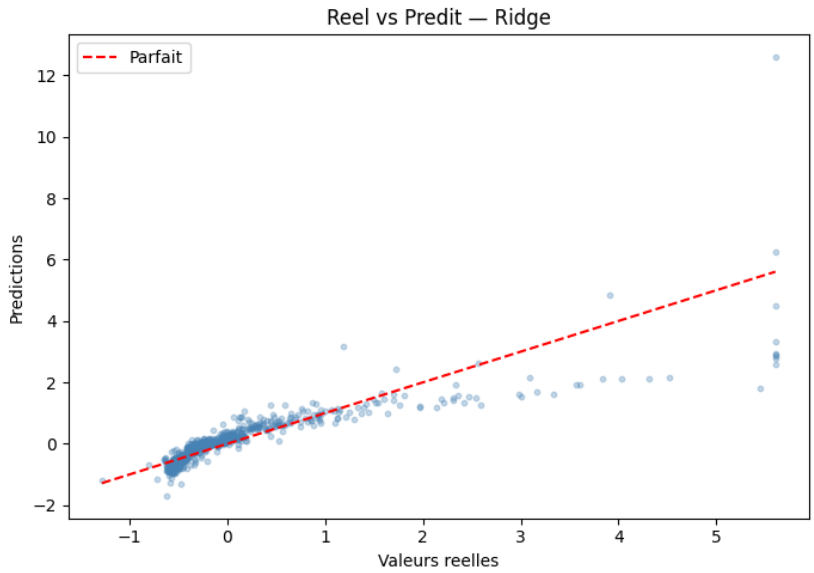


FIGURE 8 – Réel vs Prédit — Ridge Regression sur MonetaryTotal normalisé

Interprétation — Régression Ridge

- La droite de régression suit globalement la diagonale idéale (ligne rouge pointillée)
- Les prédictions sont bonnes pour les valeurs moyennes mais présentent plus d'erreur sur les valeurs extrêmes (outliers à droite)
- Ce comportement est typique d'une régression Ridge qui pénalise les coefficients extrêmes
- La régression opère sur des données normalisées (StandardScaler), d'où les axes centrés sur 0

8 Déploiement — Interface Flask

8.1 Architecture de l'application

Une interface web a été développée avec Flask pour permettre la prédiction du churn en temps réel à partir de données client saisies dans un formulaire.

```python
@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    # Construction du vecteur de features
    row = {col: 0 for col in rf.feature_names_in_}
    for key, col in FIELD_MAPPING.items():
        if key in data and col in row:
            row[col] = data[key]

    df = pd.DataFrame([row])
    X_scaled = scaler.transform(df)

    pred  = rf.predict(X_scaled)[0]
    proba = rf.predict_proba(X_scaled)[0][1]
    risk  = "Eleve" if proba > 0.7 else "Moyen" if proba > 0.4 else "Faible"

    return jsonify({
```

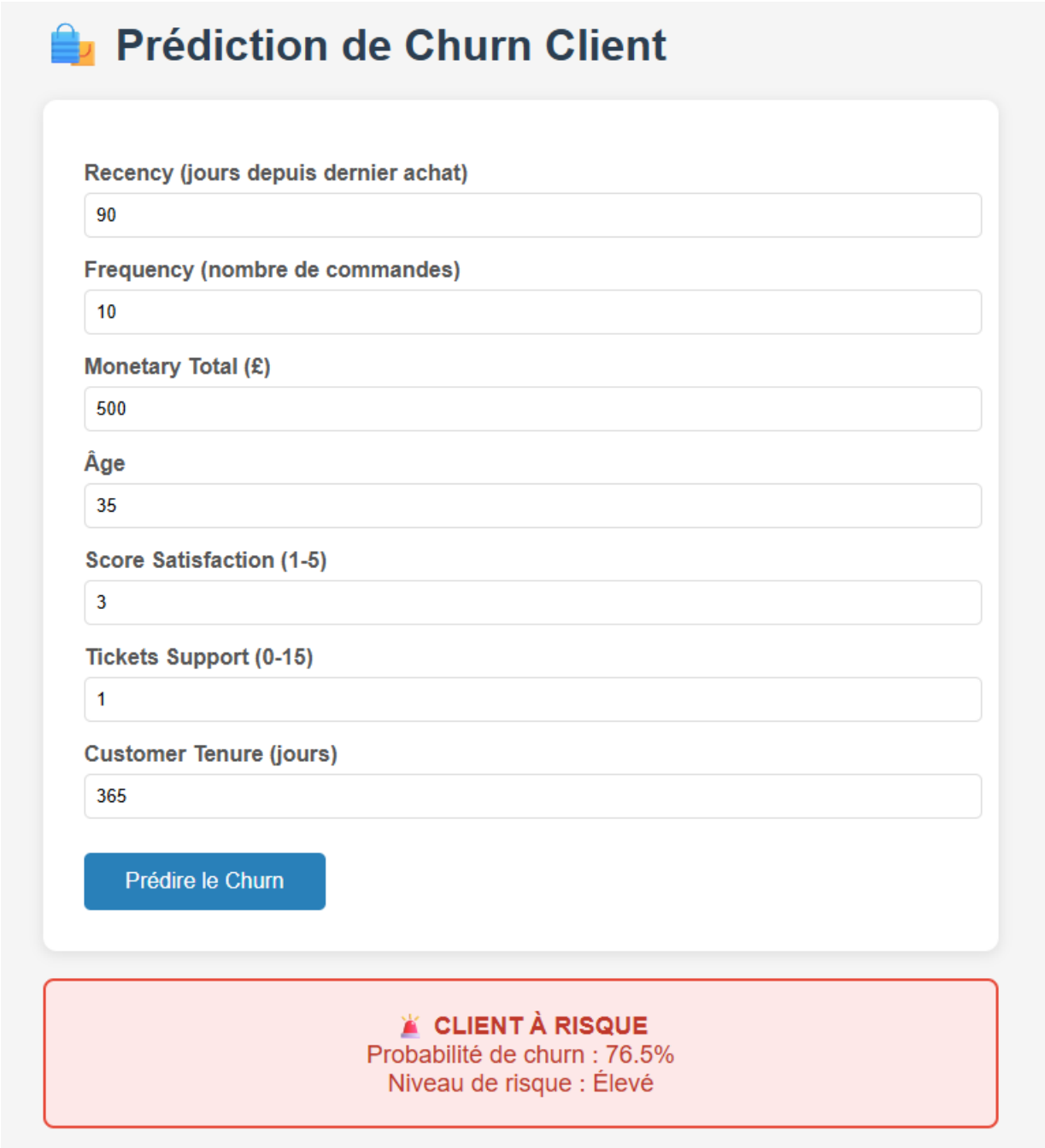
```
        "churn_prediction":  int(pred),
        "churn_probability": round(float(proba), 4),
        "risk_level": risk
    })
```

Listing 3 – Route principale de prédiction — app/app.py

8.2 Lancement de l'application

```
# Depuis le dossier racine du projet
cd app
python app.py
# Ouvrir http://localhost:5000 dans le navigateur
```

Listing 4 – Démarrage du serveur Flask



9 Conclusion et Recommandations

9.1 Bilan des modèles

Modèle	Type	Métrique	Score
Régression Logistique	Classification	AUC-ROC	0.961
Random Forest	Classification	AUC-ROC	**0.969**
Ridge Regression	Régression	R^2	–
K-Means	Clustering	4 segments	–

9.2 Recommandations métier

1. **Alerte précoce** : surveiller les clients dont `AvgDaysBetweenPurchases` augmente significativement — signal d'alerte avant le churn effectif.
2. **Saisonnalité** : les départs sont fortement liés à la saison Automne et au mois préféré. Lancer des campagnes de rétention ciblées avant les périodes à risque.
3. **Nouveaux clients** : le `LoyaltyLevel` bas est très prédictif. Renforcer l'onboarding des 90 premiers jours.
4. **Segmentation** : utiliser les 4 clusters K-Means pour des communications personnalisées : offres VIP pour le cluster actif, remises de réactivation pour le cluster dormant.
5. **Déploiement** : l'interface Flask permet une utilisation opérationnelle immédiate par les équipes marketing sans compétences techniques.

Point clé — Data Leakage

Ce projet a mis en évidence un problème critique de **data leakage** : les colonnes `ChurnRiskCategory`, `CustomerType` et `Recency` étaient des proxies parfaits du Churn, provoquant une accuracy artificielle de 100%. Leur suppression a ramené les performances à des niveaux réalistes (AUC ≈ 0.96), démontrant l'importance d'une analyse rigoureuse des données avant la modélisation.